DSST389: Advanced Data Science

Taylor Arnold
University of Richmond
tarnold2@richmond.edu

Fall 2026

Course Notes

Outline

Welcome!

This is the first semester of our year-long course in data science. Our goal of the two semesters is to introduce the core techniques of data science and how to apply them with a programming language.

The **first semester** is designed to focus on data creation, visualization, and manipulation.

The **second semester** moves towards working with increasingly complex data types such as spatial, temporal, network, text and image data. We will also continue to practice and become more comfortable with the core techniques throughout.

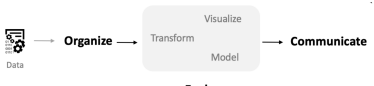
Conceptual visualization of the data science pipeline.

Instructors

This course is co-taught by Prof. Taylor Arnold (`taylor.arnold@richmond.edu`) and Prof. Sam Kellogg (`sam.kellogg@richmond.edu`).

Both of us will be in class and either of us is happy to hear from you about anything to do with the course material, the readings, the homework, or the project.

Prof. Arnold is the instructor of record, so please direct any questions about grades to him.



Conceptual visualization of the data science pipeline.

Format

The semester is broken into three units, each ending with an in-class exam, followed by a final project that runs through the end of the term. Class meetings are hands-on and interactive. Please bring a computer, pencil, and something to write on to each class. Class materials can be found on the course website.

All of the material is available on the course website: `https://taylor-arnold.github.io/courses/dsst389-s26/`

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Class Structure

The general structure of each class:

- (Before class) Do the assigned reading
- (Before class) Do the assigned homework (hand-written)
- (Start of class) Fill out class form for attendence and homework completion
- Slides for new material (5-10 minutes)
- Discussion of the homework in groups (5-10 minutes)
- Questions and/or class poll of new material (5 minutes)
- Review of last notebook (5)
- Work on current class notebook (remainder of the class, roughly ~45 minutes)

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Grading

The final grade is an evenly weight blend of five components:

- Exam I (20%)
- Exam II (20%)
- Exam III (20%)
- Final Project (20%)
- Attendance + Homework (20%)

The exams will be given in class on the dates listed. Full study guides are posted on the website.

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

External Speakers

Three candidates for a faculty position will visit campus in mid-November, and each will give a public research talk. You are required to attend all three. These are a required component of the course but are not associated with a numerical grade. The dates, times, and locations will be posted on the course website during the second week of classes.

If you have an unavoidable conflict with one or more of the talks, you must contact us *before the first of the three talks*, explain why, and identify a substitute for each talk you will miss. A substitute must be another on-campus talk with a data science component, and you will be asked to write a short justification explaining what that component was. Attendance at each talk, including substitutes, is recorded through the talk form linked on the course website.

DSST289: Introduction to Data Science

[syllabus] [dan] [book] [solutions] [sheet] [slides] [zoom] [form] [poll]

Below are the readings and homework to be done before each class. There are also links to the notebooks we will be working on together in class. Study guides for the exams will be posted below at least one week ahead of time. The dates of the external talks, of which you must attend two, are given at the bottom of this page.

A **final project** will be introducted in class on October 26th. You will design, collect, document, and analyze a dataset about your own life over a fixed fourteen-day window running from **November 2nd through November 15th**. Read the project description before Homework 15. There is no final exam.

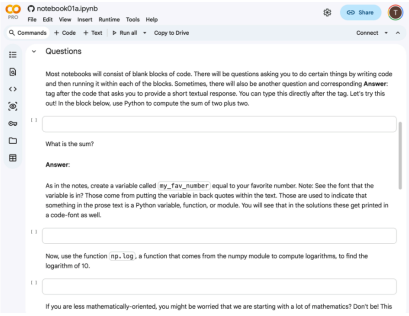
Date	Topics	Reading	Homework	Notebook
2026-08-24	Course Introduction	—	—	—
2026-08-26	Introduction to Python	1.1–1.7	Homework01	Notebook01
2026-08-31	Tabular Data	1.8–1.10	Homework02	Notebook02
2026-09-02	Organize: Sort + Select	2.1–2.6	Homework03	Notebook03
2026-09-07	Organize: Rename + Modify	2.7–2.11	Homework04	Notebook04
2026-09-09	Data Types	3.1–3.9	Homework05	Notebook05
2026-09-14	[Exam I]			
2026-09-16	Grammar of Graphics	4.1–4.3	Homework06	Notebook06
2026-09-21	Aesthetics + Geometries	4.4–4.8	Homework07	Notebook07

Screen shot from the course website. Please check website for most up-to-date details.

Google Colab

We will use the Google Colab environment for running Python. This allows us to run Python code directly from the browser for free using your university Google account with zero setup required.

Please keep in mind that Colab is simply the means of running Python. Everything we learn is directly applicable to running Python on your own machine or using a different cloud-based service.



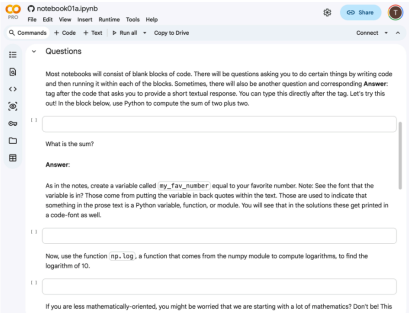
The Google Colab environment running in a web browser.

Running Code

Code in Colab is organized into **cells**. There are two ways to run the code inside a cell:

- Click the **run button** (the circle with a triangle) on the left side of the cell.
- Use a **keyboard shortcut**: `Shift+Enter` runs the cell and moves to the next one; `Ctrl+Enter` (`Cmd+Enter` on a Mac) runs the cell in place.

The keyboard shortcuts are much faster once you get used to them, so try to make a habit of using them.

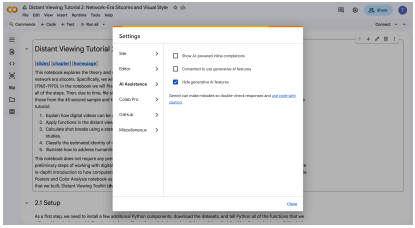


Hover over a cell to reveal the run button on its left side.

Colab Settings

Before we start, please change two sets of settings. Open the **Tools > Settings** dialog and:

1. Go to the **AI Assistance** tab. Turn off the first two options and set the third to **hide generative AI**, as in the image.
2. Go to the **Editor** tab and unselect **“Automatically close brackets and quotes in code cells”**.



The AI Assistance settings, configured as required.

Outline

Reviewing the First Semester

This chapter reviews the introductory material from the first semester (chapters 1–10). We assume you still remember the *concepts* — tidy data, units of observation, the data science pipeline. What we review here is the *code*: manipulating data and building plots.

- Three starting points in this room: some of you used R, some used pandas, and most used Polars + plotnine with slightly different conventions
- Everything this semester uses one shared style: Polars for data, plotnine for plots, all chained together with dots
- Two familiar datasets carry the examples: `country` (one row per country) and `cellphone` (one row per country per year)

Review

The Setup Code

Every notebook opens with the same imports.

`import` makes a module available under a prefix (`pl.`, `funs.`); `from ... import` pulls specific names in directly; `as` sets a short alias.

`pl.read_csv()` then loads a CSV into a *DataFrame*: one row per observation, one column per variable.

```
import polars as pl
from plotnine import ggplot, aes
from polars import col as c

import funs

country = pl.read_csv("data/countries.csv")
country
```

```
shape: (135, 15)
iso    full_name    region      pop    hdi
"SEN"  "Senegal"    "Africa"    18.93  0.53
"VEN"  "Venezuela"  "Americas"  28.52  0.709
"FIN"  "Finland"    "Europe"    5.623  0.948
...    ...          ...         ...    ...
```

Referring to Columns

A column is named with the `c.` prefix.

On its own, `c.pop` is not data — it is an *expression*: a recipe that Polars runs once per row. Attach arithmetic, a comparison, or a method to build a larger recipe.

Nothing runs until the expression is handed to a method like `.select()` or `.filter()`.

```
c.pop              # the pop column
c.hdi > 0.9        # a True/False per row
c.pop.sqrt()       # a number per row
c.pop * 1000       # a number per row

# each line describes a computation,
# not a value -- the same expression
# can be reused in any method
```

Review

Chaining Methods

Each method takes a DataFrame and returns a new one, so methods chain end to end.

Wrap the whole chain in parentheses and write one method per line. The original DataFrame is never modified.

The same `c.` expression drops into whichever method needs it — here `.filter()`, `.sort()`, and `.select()`.

```
(
    country
    .filter(c.region == "Europe")
    .sort(c.hdi, descending=True)
    .select(c.iso, c.hdi, c.happy)
    .head(n=3)
)
```

```
shape: (3, 3)
iso    hdi    happy
"ISL"  0.959  7.53
"NOR"  0.970  7.30
"CHE"  0.967  7.24
```

Review

From Chain to Plot

The chain does not stop at a table.

`.pipe(ggplot, aes(...))` hands the DataFrame to plotnine mid-chain, and every layer after it is added with a *dot* — `.geom_point()`, `.geom_text()` — not the `+` that base plotnine uses.

Columns are still named with `c.`, now inside `aes()`.

```
(
    country
    .filter(c.region == "Europe")
    .pipe(ggplot, aes(c.hdi, c.happy))
    .geom_point()
    .geom_text(aes(label=c.iso), size=6, nudge_y=0.5)
)

# same dot chain as the data steps;
# c. still names the columns, geoms
# join with . instead of +
```

Review

Verb: select and sort

`.select()` keeps only the columns you name, in the order given; `.drop()` is the inverse.

`.sort()` reorders rows by one or more columns. `descending=True` reverses the order, and later columns break ties in earlier ones.

```
(
    country
    .select(c.iso, c.region, c.pop)
    .sort(c.pop, descending=True)
    .head(n=3)
)
```

```
shape: (3, 3)
iso    region      pop
"IND"  "Asia"      1464
"CHN"  "Asia"      1408
"USA"  "Americas"  347.3
```

Review

Verb: filter

`.filter()` keeps rows where a condition is `True`; rows where the test is null are dropped as well.

Combine conditions with `&` (and) and `|` (or), wrapping each in parentheses. `.is_in()` tests membership in a set of values.

```
(
    country
    .filter(
        (c.region == "Europe") & (c.hdi > 0.9)
    )
)
```

```
(
    country
    .filter(c.region.is_in(["Europe", "Africa"]))
)
```

```
# first: European rows with a very
# high HDI
# second: every European OR African row
```

Review

Verb: with_columns

`.with_columns()` adds or overwrites columns and keeps the rest of the table.

Name each new column with `=` and build it from a `c.` expression. `pl.when().then().otherwise()` builds a column from a condition; wrap literal text in `pl.lit()`.

```
(
    country
    .with_columns(
        pop_1k = c.pop * 1000,
        zone = (
            pl.when(c.lat.abs() < 23.5)
              .then(pl.lit("tropical"))
            .otherwise(pl.lit("temperate"))
        )
    )
)

# pop_1k and zone are appended;
# all 15 original columns remain
```

Review

Verbs: group_by and agg

`.group_by()` splits rows into groups; `.agg()` then computes one row of summaries per group.

Only the grouping column(s) and the summaries you ask for survive; everything else is dropped. `pl.len()` counts the rows in each group.

```
(
    country
    .group_by(c.region)
    .agg(
        hdi_avg = c.hdi.mean(),
        count = pl.len()
    )
)
```

```
shape: (5, 3)
region      hdi_avg  count
"Africa"    0.5834   37
"Americas"  0.7836   19
"Europe"    0.8989   38
"Asia"      0.7609   39
"Oceania"   0.948    2
```

Review

Verb: join

A `.join()` combines two tables along a shared key. Here `border` lists neighboring-country pairs and `c_sml` holds one Gini value per country.

Called on the left table, it takes the right table and the key column. The default `inner` drops non-matching rows; `how="left"` keeps every left row, filling `null` where the right side has no match.

```
(
    border
    .join(c_sml, on=c.iso)
)
```

```
shape: (641, 3)
iso  iso_neighbor  gini
AFG  IRN           27.8
AFG  PAK           27.8
...  ...           ...
ZWE  ZMB           50.3

# how="left" would keep all 829 rows,
# with null gini where iso didn't match
```

Review

Verbs: pivot and unpivot

`.pivot()` turns a long table wide: `index` is the new row unit, `on` supplies the new column names, and `values` fills them in.

`.unpivot()` reverses it, collapsing many columns back into two — `variable` and `value` by default, renamed here with `variable_name` and `value_name`.

```
cell_wide = cellphone.pivot(
    index=c.year, on=c.iso, values=c.cell
)

cell_wide.unpivot(
    index=c.year,
    variable_name=c.iso, value_name=c.cell
)

# pivot: (3480, 3) -> (20, 175)
# one column per country

# unpivot: back to (3480, 3)
year  iso    cell
2003  "AFG"  0.8798
2004  "AFG"  2.547
...   ...    ...
```

Review

Plot Geometries

Every plot starts the same way: pipe a DataFrame into `ggplot` with an `aes()` mapping columns to position, color, or size, then chain each geometry with a dot.

- One mark per row: `.geom_point()`, `.geom_text()` (needs a `label`), `.geom_line()` (ordered along *x*), `.geom_col()` (a bar per row)
- Statistical geometries run a hidden group-and-count first: `.geom_bar()` (counts a category), `.geom_histogram()` (bins a number), `.geom_boxplot()`
- A *fixed* aesthetic (same for every row) goes outside `aes()`; a *mapped* one (varies by row) goes inside. `.coord_flip()`, `labs()`, and `theme_*()` adjust axes, labels, and styling

Review

Window Functions: over

A window function computes a group-level number but keeps every row, writing the group's answer onto each — the counterpart to `.agg()`, which collapses.

`.over()` scopes the computation to a group. Read `c.pop.sum().over(c.region)` inside out: a sum, scoped to each region.

```
(
    country
    .with_columns(
        pop_share = c.pop / c.pop.sum().over(c.region) * 100
    )
    .sort(c.pop_share, descending=True)
)

# ordered windows -- .shift(), .rank(),
# running totals -- also scope with .over()
```

```
shape: (135, 16)
iso  region      pop_share
AUS  "Oceania"   83.70
USA  "Americas"  35.65
IND  "Asia"      30.93
CHN  "Asia"      29.92
...  ...         ...
```

Review

The Core Toolkit

Everything above is the toolkit the rest of the semester builds on.

- Single-table verbs — `.filter()`, `.select()`/`.drop()`, `.sort()`, `.with_columns()` — each return a new table, so they chain
- Group and reshape — `.group_by().agg()` collapses to one row per group; `.pivot()`/`.unpivot()` move between long and wide; `.join()` combines tables on a key
- Keep the group *and* the rows with `.over()`; plot by piping into `ggplot` and chaining geometries with dots
- New data types ahead — spatial, temporal, text, network, image — mostly just add new columns and a few specialized methods on top of these same verbs

Outline

Outline

Spatial Data

Every dataset we have used so far has been detached from physical space; a lot of real data is not. Crime reports happen at street addresses, and hospitals sit a measurable distance from every school in a city.

- A *geometry* attaches a location or region to a row: a single point, or a polygon with thousands of vertices
- Once rows have geometries, we can ask questions with no ordinary-table equivalent: which points fall inside which polygon? which polygons touch? what is nearest?
- Polars has no native geometry support; the standard library is GeoPandas, built on pandas

The geo Namespace

Importing `funs` registers a namespace called `geo` on every Polars DataFrame, bridging Polars and GeoPandas without manual conversion.

- Any GeoPandas method or property is reachable through it: `metro.geo.to_crs(5069)`, `state.geo.area`, `metro.geo.sjoin(state)`
- Each call converts the table to GeoPandas, runs the operation, and converts the result back to Polars
- Geometries are stored as plain text (WKT) in a `geometry` column; the coordinate system is tracked in a `_crs` column

Building Points

`points_from_xy` builds `geometry` from two coordinate columns and records the coordinate system.

It defaults to WGS 84 (`crs=4326`), ordinary latitude and longitude, the same system GPS devices use.

```
metro = metro.geo.points_from_xy("lon", "lat")
metro
```

```
shape: (934, 15)
name        lon     lat    geometry                  _crs
"New York"  −74.10  40.77  "POINT (−74.10 40.77)"   "EPSG:4326"
"Los Ang."  −118.1  34.22  "POINT (−118.1 34.22)"   "EPSG:4326"
"Chicago"   −87.96  41.70  "POINT (−87.96 41.70)"   "EPSG:4326"
...         ...     ...    ...                       ...
```

Plotting Raw Coordinates

Because longitude and latitude are just numbers, a basic scatter plot needs no spatial tools at all: longitude on *x*, latitude on *y*.

This treats a curved surface as an ordinary Cartesian grid, so distortion worsens at higher latitudes. There is also nothing to orient by — no coastlines, no borders.

US metro areas plotted directly from longitude and latitude.

Coordinate Reference Systems (CRS)

A *coordinate reference system* (CRS) defines how coordinates map to locations on Earth's curved surface. Flattening a globe always trades off shape, area, or distance.

- WGS 84 (`EPSG:4326`): degrees of latitude/longitude, used by GPS — the default we start from
- A *projected* CRS is optimized for a region or purpose, e.g. `EPSG:5069` (NAD83 Conus Albers) for the contiguous US
- Look up codes at EPSG.io or spatialreference.org

Reprojecting and Mapping

`.to_crs()` transforms coordinates into a new CRS.

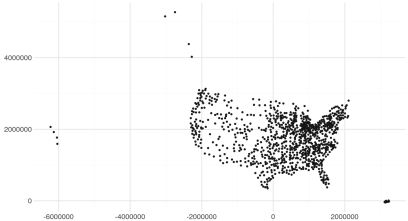
`geom_map()` draws geometries straight from the `geometry` column; `coord_fixed()` keeps one map unit the same length on both axes.

```
(
    ggplot(metro.geo.to_crs(5069))
    .geom_map()
    .coord_fixed()
)
```

Projected Points

Reprojected into Albers, the point cloud now looks like the familiar outline of the continental US.

Alaska and Hawaii appear distorted, since this projection is optimized for the lower 48 states.



The same metro points after reprojecting to EPSG:5069.

Spatial Polygons

Many phenomena are better described by *polygons*: closed shapes bounding a region. States, zip codes, census tracts, and watersheds are all naturally polygons.

- GeoJSON is a common polygon format: JSON-based, standardized as RFC 7946; Shapefiles, GeoPackage, and PostGIS are also common
- `funs.read_file()` reads most spatial formats, returning a Polars DataFrame with WKT geometries
- Each polygon's `geometry` entry lists every vertex of its boundary — sometimes thousands for complex coastlines

Loading and Mapping Polygons

Polygon data cannot be meaningfully drawn with a scatter plot; `geom_map()` traces each shape from its vertex coordinates instead.

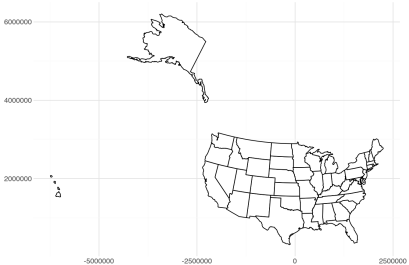
The effect of projection is far more noticeable with polygons than with points.

```
state = funs.read_file("data/acs_state.geojson")
(
    ggplot(state.geo.to_crs(5069))
    .geom_map(fill="white", color="black")
    .coord_fixed()
)
```

State Boundaries

Fifty states plus DC, drawn from their GeoJSON polygons after projecting to EPSG:5069.

The Albers projection compresses and expands the coordinates to give a faithful picture of true shapes and relative sizes.



US state boundaries, projected and drawn with `geom_map`.

Centroid and Area

`centroid` and `area` are only meaningful in a projected CRS; on raw latitude/longitude they come out as nonsense.

`centroid` replaces each polygon with its geometric center; `area` appends a numeric column, in square meters here.

```
state = state.geo.to_crs(5069)
state.geo.centroid
state = state.geo.area
state.sort(c.area).select(c.name, c.area)
```

```
shape: (50, 2)
name          area
"Rhode Isl."  2.76e9
...           ...
"California"  4.09e11
"Texas"       6.88e11
"Alaska"      1.46e12
```

```
# area in square meters
```

Choropleth Maps

A *choropleth map* colors regions by the value of a variable.

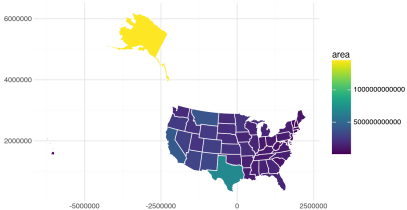
With `area` already a column, building one takes a single aesthetic mapping: `fill=c.area`.

```
(
    ggplot(state, aes(fill=c.area))
    .geom_map(color="white")
    .coord_fixed()
)
```

Choropleth Map

Alaska dwarfs the other states by area; Texas and California follow.

Larger regions draw the eye even when they matter less, so choropleths work best for area-based variables (density) rather than raw counts (population).



States colored by land area.

Spatial Joins

Ordinary joins match rows by equal keys. A *spatial join* matches rows by geometric relationship instead.

- `.geo.sjoin()` tests a *predicate* between geometries in two tables: `intersects`, `contains`, `within`, `touches`, `crosses`, `overlaps`
- Both tables must share a CRS before joining
- Row order matters: the left table's geometry survives in the output, so joining points into polygons keeps points; reversing keeps polygons, duplicated once per match

Spatial Join

The default predicate matches points to the polygon that contains them.

Column names shared by both tables get `_left`/`_right` suffixes; a metro straddling a state line is assigned to whichever state contains its center point.

```
metro = metro.geo.to_crs(5069)
metro.geo.sjoin(state)
```

```
shape: (920, 20)
name_left     name_right
"New York"    "New Jersey"
"Los Angel."  "California"
"Chicago"     "Illinois"
"Dallas"      "Texas"
"Houston"     "Texas"
...           ...

# New York's CENTER POINT falls just
# across the river, in New Jersey
```

Nearest Neighbor Join

`.sjoin_nearest()` finds, for each row, the closest row in a second table — e.g. the nearest hospital to each school.

`exclusive=True` stops a row from matching itself; `distance_col` records the distance, in CRS units.

```
metro.geo.sjoin_nearest(
    metro, exclusive=True, distance_col="distance"
)
```

```
shape: (934, 30)
name_left     name_right  distance
"New York"    "Trenton"   74171
"Los Angel."  "Oxnard"    89324
"Chicago"     "Kankakee"  63507
"Dallas"      "Sherman"   90905
"Houston"     "El Campo"  97541
...           ...         ...

# distance in meters (Albers CRS)
```

Buffers

A *buffer* expands a geometry outward by a fixed distance, turning points into discs.

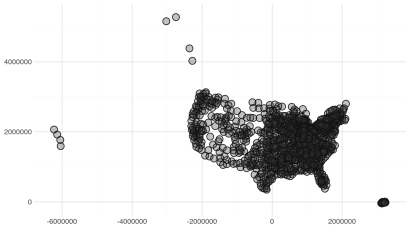
Buffers answer “within *X* of” questions; joined against another table, they find every feature near a point, not only the one containing it.

```
(
    ggplot(metro.geo.buffer(100_000))
    .geom_map(alpha=0.3)
    .coord_fixed()
)
```

Buffers Around Metro Areas

Each metro point becomes a 100 km disc; overlaps pile up where discs are drawn on top of each other.

The overlaps trace a nearly solid band along the eastern seaboard and thin to isolated circles across the Mountain West — a picture of urban density the point map only hinted at.



100 km buffers around each metro area.

Dissolving Boundaries

`.dissolve()` is the spatial counterpart of `group_by`/`agg`: it merges each group's geometries into one and erases the internal boundaries.

`by` names the group key; `aggfunc` says how to combine the ordinary columns.

```
region = state.geo.dissolve(
    by="side", aggfunc={"area": "sum"}
)
```

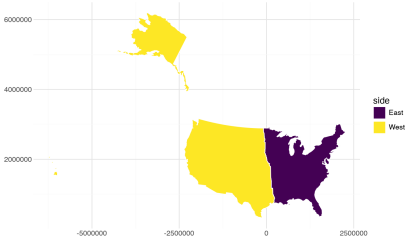
```
shape: (2, 4)
side    area     geometry
"East"  3.06e12  "POLYGON ((...))"
"West"  6.21e12  "MULTIPOLYGON (((...)))"

# West is a MultiPolygon: Alaska and
# Hawaii don't touch the mainland
```

Dissolved Regions

States grouped into East/West by their centroid, then dissolved into two merged polygons.

The western geometry is a MultiPolygon, since Alaska and Hawaii do not touch the mainland.



State polygons dissolved into two regions.

Best Practices

Spatial data introduces error sources that do not arise with ordinary tables.

- **CRS mismatches**: operations on datasets in different CRSs silently give wrong locations or miss real matches — check and align with `.to_crs()` first
- **Unprojected calculations**: never compute distance or area on raw latitude/longitude; project first
- **Invalid polygons**: self-intersections and other topological errors can make functions fail or misbehave
- **Boundary effects**: a feature straddling a border is assigned by a single representative point — handle these edge cases deliberately

Outline

Data APIs

Every dataset we have used so far arrived as a file. A lot of real data is not distributed that way — it lives on remote servers and is only reachable through a programmatic interface.

- An *API* (application programming interface) exchanges structured data, usually JSON, over HTTP — the same protocol your browser uses
- Two goals: learn the mechanics (requests, status codes, pagination, nested JSON) and use them to build something real
- Case study: a corpus of Wikipedia articles on 19th-century British and Irish novelists, reused in later chapters

HTTP Requests

- Each request targets an *endpoint* URL with a set of *parameters* attached
- GET retrieves data; POST/PUT/DELETE modify it — data collection almost always uses GET
- A numeric *status code* reports what happened: 200 success, 404 not found, 429 rate limited, 5xx server error
- Many APIs require an account and key; the Wikimedia APIs used here require neither

A First Request

An API call needs an endpoint, a dictionary of parameters, and headers identifying who is calling.

The `User-Agent` header lets server administrators reach you if your requests cause problems.

```
params = {
    "action": "query",
    "format": "json",
    "prop": "info",
    "titles": "Emily Brontë"
}
response = session.get(
    base_url, params=params, headers=headers
)
response.status_code
```

```
200
```

```
# data = response.json()
# data["query"]["pages"] looks like:
# {'9810': {'pageid': 9810,
#   'title': 'Emily Brontë',
#   'touched': '2026-07-12T06:01:37Z',
#   'length': 72283, ...}}
```

Flattening Nested JSON

A JSON response parses directly into nested Python dicts and lists. The useful data usually sits several levels deep, keyed oddly — here by numeric page ID rather than as rows.

We navigate to it, pull the fields we need into a list of dicts, and hand that list to Polars.

```
rows = []
for page in data["query"]["pages"].values():
    rows.append({
        "page_id": page["pageid"],
        "title": page["title"]
    })
pl.DataFrame(rows)
```

```
shape: (1, 2)
page_id  title
9810     "Emily Brontë"
```

Caching, Pagination, and Etiquette

- `requests_cache` saves every response to disk; reruns hit the cache instead of the network — the cache becomes a permanent *raw layer*
- A single response often caps out (e.g. 500 results); a `continue` block in the JSON says how to fetch the next page
- Rate limits cap requests per unit time; exceeding them returns 429 — send a real `User-Agent`, request serially, and pause between calls

Defining the Population

- The first and most consequential decision: which pages belong in the corpus?
- Options: hand-curate a list, query a knowledge graph, or use Wikipedia's category system
- Category membership reflects editor tagging, not ground truth — an author missing the tag is invisible to us
- Every population definition should be written down where later users of the data can find it

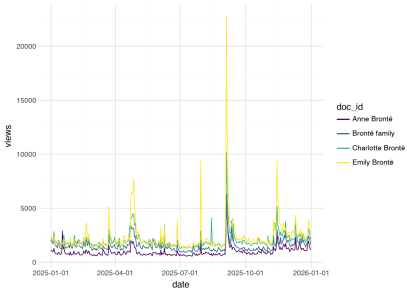
Stable vs. Readable Keys

- Titles are the least stable identifier — pages get renamed, and yesterday's title becomes today's *redirect*
- The numeric `page_id` survives renames; the title (`doc_id`) stays human-readable — every table in the corpus carries both
- Redirects and missing titles must be resolved and logged, never silently dropped

Page View Statistics

Page views come from a second, REST-style API: the request path itself encodes project, access type, article, and date range, rather than a parameter dictionary.

Daily views for the three Brontë sisters show weekly rhythms and occasional spikes tied to real-world attention.



Daily Wikipedia page views for the Brontë sisters, 2025.

Raw and Processed Layers

- *Raw layer*: the requests cache — exact, unmodified server responses
- *Processed layer*: tidy Parquet tables built from those responses
- Every processed table can be rebuilt from the raw layer without touching the network, so downstream decisions can be revisited later
- A short manifest records what was collected, from where, and under what population definition

Reproducible Snapshots

- Wikipedia changes constantly — what we collect is a *snapshot*, not a fixed dataset
- Revision IDs pin an article to one immutable version, recoverable from the API forever
- Reproducibility here means being precise about what was captured, not freezing the source

Outline

Temporal Data

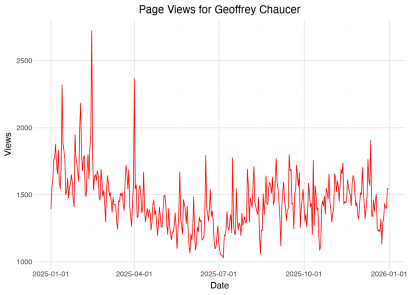
Any dataset with a date or time column, or whose rows simply have a meaningful order, is temporal. Two tables from the Wikipedia corpus qualify: daily page views and a 500-edit revision history per author.

- Window functions (`.shift()`, `.over()`) and conditional joins (`.join_asof()`, `.join_where()`) return here, now on data with real clocks
- New to this chapter: the temporal type system itself — dates, datetimes, time zones, truncation, durations

Real Dates, Not Row Numbers

Plotting views against a row number or a fractional year works, but the axis needs explaining and says nothing a reader can act on.

A real `Date` column gives plotnine sensible, labeled breaks for free — and turns an anonymous spike into a fact: Chaucer's biggest day is always 14 February, thanks to *Parlement of Foules*.



Chaucer's page views plotted against a real date axis.

Date and Datetime Objects

`pl.date()` builds a `Date` from year/month/day columns; `.str.to_datetime()` parses an ISO-8601 string into a `Datetime`.

Both types unlock the `.dt` namespace: weekday, hour, quarter, and more, all as portable numbers rather than locale-dependent names.

```
chaucer = page_views.with_columns(
    date = pl.date(c.year, c.month, c.day)
)
page_revisions = page_revisions.with_columns(
    timestamp = c.timestamp.str.to_datetime()
)

chaucer.schema["date"]
# Date

chaucer.select(c.doc_id, c.date, c.views).head(3)
# "Geoffrey Chaucer"  2025-01-01  1392
# "Geoffrey Chaucer"  2025-01-02  1553
# "Geoffrey Chaucer"  2025-01-03  1609

page_revisions.schema["timestamp"]
# Datetime(time_unit='us', time_zone=None)
```

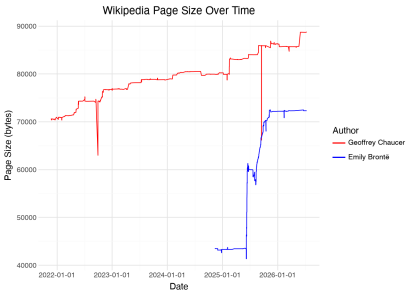
Truncating Dates

- `.dt.truncate("1h")` or `.dt.truncate("1d")` rounds a timestamp down, discarding finer precision
- Truncation turns a continuous timestamp into a categorical bin — the standard first step before `.group_by()` on an event log
- `.dt.round()` rounds to the nearest unit instead of down; `.dt.date()` extracts just the date part of a datetime

Observation Windows Differ

We collected each author's 500 most recent edits, not a fixed span of years, so the window covered differs by author.

Emily Brontë's page is edited often, reaching back only to late 2024; Chaucer's moves slowly, reaching back to 2021 — comparing the two as though they spanned the same years would be a mistake.



Article size over time for two authors with very different edit rates.

Time Zones: Naive vs. Aware

- A *naive* datetime carries no time zone; an *aware* one is tagged with a specific zone
- `.dt.replace_time_zone()` attaches a zone — a claim about what the clock reading always meant
- `.dt.convert_time_zone()` re-expresses an already-aware instant in a different zone — an arithmetic operation
- Converting a naive column silently assumes it was UTC; attach first, then convert

Window Functions Revisited

The same two ideas from window functions apply unchanged, now on irregularly spaced timestamps.

- `.diff().over(c.doc_id)` gives the size change of each edit — how many bytes it added or removed
- Reaching back two rows with `.shift(2)` instead of one flags *reversions*: edits that put an article back where it stood two edits earlier
- `.over()` is still required — ordered functions run past group boundaries without it

Rolling Windows in Time

A row-count window (`.rolling_mean()`) assumes evenly spaced rows, which edit logs are not: two edits might be minutes or months apart.

`.rolling_mean_by()` defines the window as a *span of time* instead of a row count, using the `by` timestamp column.

```
(
    page_revisions
    .filter(c.doc_id == "Geoffrey Chaucer")
    .sort(c.timestamp)
    .with_columns(
        size_smooth = c.size.rolling_mean_by(
            by=c.timestamp, window_size="30d"
        )
    )
)
```

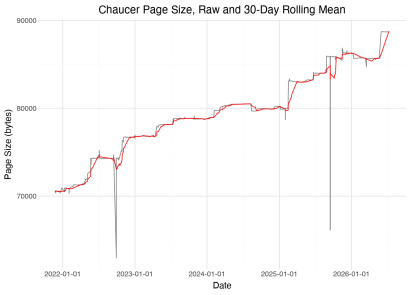
```
shape: (500, 8)
timestamp           size   size_smooth
2021-11-23 15:31:56  70572  70572.0
2021-11-23 15:35:22  70575  70573.5
2021-11-23 15:37:01  70571  70572.7
...                  ...    ...
2026-07-10 23:37:58  88794  88735.0
```

```
# no .over() needed — already
# filtered to one author
```

Time-Aware Smoothing

The grey line is Chaucer's raw article size after every edit, jagged with vandalism and its reversal.

The red line is the 30-day rolling mean, which shows the article's actual trajectory rather than every transient edit.



Chaucer's page size, raw and smoothed with a 30-day time-based window.

Joining on Time

- `.join_asof()` matches each row to the *nearest* value in another table — built for exactly this: pairing a revision with the most recent daily view count `strategy="backward"`
- `by="doc_id"` requires an exact match before the nearest-in-time search, so authors are never matched across each other's data
- `.join_where()` keeps every pair satisfying an inequality — e.g. which authors' lifetimes overlapped — producing a network of shared lifetimes

Outline

Textual Data

Every dataset so far has been structured: rows with well-defined numeric, categorical, or temporal columns. A large share of recorded information exists only as unstructured text.

- A sentence has grammar, and meaning depends on context — “bank” means something different in “river bank” than in “bank account”
- This chapter turns unstructured text into structured tables of tokens, counts, and annotations, so the familiar filter/group/join toolkit applies
- Documents: Wikipedia pages for the same 75 British and Irish authors studied in the temporal data chapter

The NLP Pipeline

spaCy provides pre-trained models that tokenize text and annotate it linguistically.

- `DSText.process` sends each document through tokenization, part-of-speech tagging, lemmatization, named entity recognition, and dependency parsing
- Underneath, it is three nested loops — documents, then sentences, then tokens — building one dictionary per token and stacking them into a Polars DataFrame
- `tid` and `head_idx` are renumbered within each sentence, which is what makes join-based analysis of grammar possible later

Token Annotations

Each row of `anno` is one token: a word or punctuation mark, labeled with its lemma, part of speech, and grammatical role.

Key columns: `doc_id`, `sid`, `tid` (a primary key), `lemma`, `upos`, `dep`, `head_idx`, `ent_type`.

```
anno = DSText.process(docs, nlp)
anno
```

```
shape: (408700, 15)
```

doc_id	sid	tid	token	lemma	upos	dep	head_idx	ent_type
"Marie.."	1	1	Marie	Marie	PROPN	compound	3	PERSON
"Marie.."	1	2	de	de	X	nmod	3	PERSON
"Marie.."	1	3	France	France	PROPN	nsubj	4	PERSON
"Marie.."	1	4	was	be	AUX	ROOT	4	""
"Marie.."	1	5	a	a	DET	det	6	""
"Marie.."	1	6	poet	poet	NOUN	attr	4	""
...	...	...	...	...	...	...	...	...

Word Frequencies

Once text is a table, ordinary Polars operations answer linguistic questions with no text-specific tools at all.

- Filtering `upos == "NOUN"` and grouping by `lemma` surfaces the corpus's most common nouns: *year*, *life*, *poem*, *poet*
- Grouping by `[doc_id, lemma]` instead gives the top adjectives on *each* author's page — "Victorian", "Gothic"
- Generic words dominate every page's list, a limitation the next chapter's TF-IDF weighting addresses

Building N-grams

Meaning often lives in word combinations — “New York”, “poet laureate” — not single tokens.

The exact `.shift()/.over()` pattern from the temporal data chapter pairs each token with its neighbor: a text read token-by-token is just another ordered dataset.

```
bigrams = anno.filter(c.is_alpha).with_columns(
    next_token = c.lemma.shift(-1)
        .over([c.doc_id, c.sid])
)
```

```
shape: (380244, 16)
sid  tid  lemma   next_token
1    1    Marie   de
1    2    de      France
1    3    France  be
1    4    be      a
1    5    a       poet
1    6    poet    possibly
```

Collocations via PMI

- Raw bigram counts surface generic phrases — “of the” appears often simply because both words are common
- *Pointwise mutual information* compares an observed pairing to what independence would predict: $\mathrm{PMI}(w_1, w_2) = \log_2 \frac{P(w_1, w_2)}{P(w_1)\,P(w_2)}$
- High-PMI bigrams are proper names and domain-specific terms; a minimum count filter (e.g. ≥ 5) keeps rare-word noise out

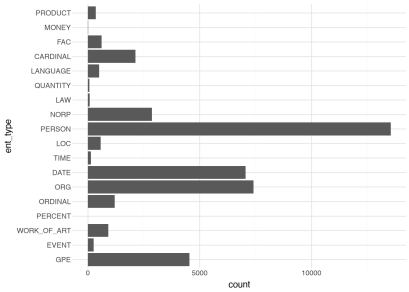
Named Entity Recognition

- spaCy tags proper nouns and specific references: PERSON, ORG, GPE, DATE, WORK_OF_ART, and more, stored in `ent_type`
- Multi-word entities (“United Kingdom”) repeat the same label on every token — `ent_iob` marks “B” for the first token of each entity
- A cumulative sum of `ent_iob == "B"` within each document assigns every entity an id, so grouping and `str.join` reconstruct full entity text

Entity Types Across the Corpus

Biographical pages are rich in names, organizations, places, and dates — unsurprising for encyclopedia entries about people's lives and work.

A different corpus (press releases, scientific abstracts) would produce a very different profile, which is what makes this distribution a useful fingerprint of a collection.



Counts of named entity types across the author corpus.

Dependency Parsing

Named entities say *what* is mentioned; dependency parsing says *how* words relate grammatically.

- Every token has a *head* and a *dependency relation* (`nsubj`, `dobj`, `amod`, …) describing how it modifies or depends on another token
- Since `head_idx` shares `tid`'s coordinate system, matching a subject to its verb is an ordinary join on `[doc_id, sid, tid]`
- The result: authors "write", "publish", "die"; works "appear", "receive", "influence"

KWIC and Complexity Metrics

- *Keyword in context* (`DSText.kwic`) prints every occurrence of a word with its surrounding tokens — the cheapest way to check what a word is really doing, when counts and scores can only summarize
- Style metrics reduce a whole document to a number: mean sentence length, type-token ratio (vocabulary richness), mean word length, function-word ratio
- All four are ordinary `group_by` + `agg` chains over `anno` — no new machinery beyond what the rest of the book already taught

Outline

From Words to Whole Documents

The previous chapter analyzed language *within* documents, one token at a time. Comparing documents to each other — which pages are most similar, do they cluster into groups — requires representing each document as a single mathematical object.

- Step 1: summarize each document by its term counts, weighted by TF-IDF to emphasize distinctive vocabulary
- Step 2: treat those weighted counts as coordinates, turning each document into a *point in space*
- Step 3: use that geometry to measure distance, project to two dimensions, and compare corpora across languages

TF-IDF

Raw term counts favor generic words (“the”, “year”, “life”). *Term frequency–inverse document frequency* (TF-IDF) downweights terms that are common across the whole collection:

- $\text{tfidf} = \text{tf} \times \left(\log\left(\frac{N+1}{\text{df}+1}\right) + 1\right)$, where *tf* is the term frequency, *df* the number of documents containing the term, and *N* the total document count
- Words appearing in nearly every document (e.g. “publisher”) score low even if frequent; words distinctive to a single page score high
- Top TF-IDF terms per document are proper nouns and domain vocabulary — exactly what a human summary would pick out

Computing TF-IDF

`DSText.compute_tfidf` is a single Polars chain: count lemmas per document, normalize by document length for *tf*, count documents per lemma for *df*, then combine.

`min_df`/`max_df` filter out lemmas that are too rare or too common, acting like an automatic stopword list.

```
tfidf = DSText.compute_tfidf(
    anno, min_df=0.01, max_df=1.0
)
```

```
shape: (106352, 7)
doc_id          lemma   tf   idf    tfidf
"A. A. Milne"   Milne   52   4.638  0.0949
"A. A. Milne"   Pooh    40   4.638  0.0730
"A. A. Milne"   the     183  1.0    0.0720
"A. A. Milne"   Winnie  21   4.638  0.0383
...             ...     ...  ...    ...

# highest-tfidf terms are the ones
# that make this page distinctive
```

Documents as Vectors

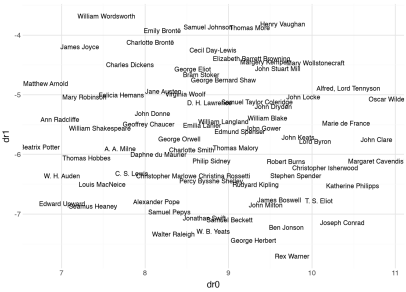
Each row of the TF-IDF table is a list of numbers — one per term. Treat that list as coordinates, and every document becomes a *point in space*.

- Two terms give a 2-D scatterplot; V terms give a point in V-dimensional space, $\mathbf{x}_d = (\text{tfidf}(t_1, d), \ldots, \text{tfidf}(t_V, d))$
- Documents close together use similar vocabulary in similar proportions; documents far apart use different language
- Most entries are zero — a document uses only a sliver of the vocabulary — giving a *sparse* representation

Mapping the Corpus with UMAP

Thousands of dimensions can't be plotted directly, but UMAP projects the document vectors down to two dimensions while preserving distances as well as possible.

Authors who use similar vocabulary land near each other — proximity on the map reflects similarity in language, not chronology or genre labels.



UMAP projection of 75 author pages built from TF-IDF vectors.

Document Distances

Euclidean distance is a poor fit for document vectors: a long page and a short page on the same topic use similar *proportions* of vocabulary but sit far apart simply because their raw counts are bigger.

- The fix: measure the *angle* between vectors instead of the straight-line gap — length no longer matters, only direction
- $d(\mathbf{x}_a, \mathbf{x}_b) = \arccos\left(\frac{\mathbf{x}_a \cdot \mathbf{x}_b}{\|\mathbf{x}_a\| \, \|\mathbf{x}_b\|}\right)$, ranging from 0 (identical proportions) to $\pi/2$ (no shared vocabulary)
- The related *cosine similarity* reports the quantity inside the arccosine directly: 1 means identical, 0 means no overlap

Computing Distances

`DSText.compute_distances` pivots the long TF-IDF table to a wide matrix, rescales every row to unit length, then computes all pairwise angles at once with a single matrix product.

The result is a long table with one row per pair — each unordered pair of documents appears exactly once.

```
text_dist = DSText.compute_distances(
    anno, min_df=0.01, max_df=0.5
)
```

```
shape: (2775, 3)
doc_id          doc_id2                   distance
"A. A. Milne"   "Alexander Pope"          1.5624
"A. A. Milne"   "Alfred, Lord Tennyson"   1.5580
"A. A. Milne"   "Ann Radcliffe"           1.5624
"A. A. Milne"   "Beatrix Potter"          1.5484
…               …                         …
```

```
# 75 authors -> C(75,2) = 2775 pairs
```

Nearest Neighbors

With distances in a table, ordinary Polars answers similarity questions directly — no special “similarity search” tool required.

- Sorting by `distance` within a single `doc_id` finds the closest matches to one page (e.g. Jane Austen)
- `.group_by(doc_id).head(1)` after sorting finds the single nearest neighbor of *every* document at once
- Worth reading against literary history: contemporaries matching contemporaries is expected; the surprises are often the most interesting rows

Across Languages

Wikipedia's French edition lets us ask whether the same authors are described differently across languages and editor communities.

- Swap in a French spaCy model (`fr_core_news_sm`) — every downstream tool (TF-IDF, vectors, UMAP) applies unchanged
- English and French vectors live in different vocabularies and can't be compared directly, but the *structure* of each collection can
- A French UMAP projection that groups authors differently reflects a genuine difference in framing, not just translation

From Geometry to Networks

- TF-IDF turns token counts into distinctive-vocabulary scores; treating those scores as coordinates turns documents into points in space
- UMAP gives a two-dimensional map; angle distance gives an explicit number for every pair of documents
- Next chapter: a table of pairwise distances *is* a network — connect documents below a distance threshold, or link each to its nearest neighbors

Outline

Network Data

A network is a set of objects (*nodes*) plus a set of relationships between pairs of them (*edges*). What counts as a node and what counts as an edge is a modeling choice, and that choice shapes everything downstream.

- An *edge list* — one row per edge, with a starting and ending id — is enough to describe an entire network as an ordinary table
- *Directed* networks distinguish the start from the end of an edge (A links to B); *undirected* networks only ask whether a connection exists
- Any directed network can be viewed as undirected by dropping direction; the reverse is not true, so undirected is the safer starting point

Creating a Network Object

`DSNetwork.process` turns an edge list into a `node` table, an `edge` table, and an `igraph` graph object, computing layout coordinates and centrality metrics along the way.

The `node` table's `x`/`y` columns are a *graph drawing* layout — like PCA or UMAP, individual values don't mean anything; only relative position does.

```
node, edge, G = DSNetwork.process(
    page_citation, directed=False
)
```

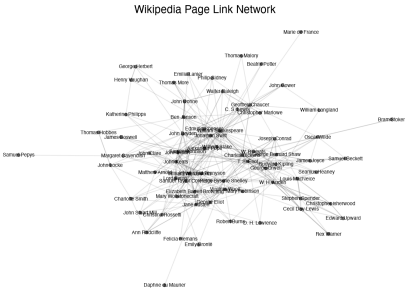
```
shape: (72, 10)
id                    x      y     degree  eigen  between
"Marie de France"     -3.66  4.29  1       0.024  0.0
"Geoffrey Chaucer"    -1.38  1.71  16      0.409  147.1
"William Langland"    -3.64  1.68  3       0.062  4.72
"John Gower"          -2.44  2.46  5       0.119  7.74
"John Dryden"         0.97   1.14  25      0.731  197.5
...                   ...    ...   ...     ...    ...

# 377 links, 75 authors -> 72 nodes
```

Visualizing the Network

`geom_segment` draws the `edge` table's `x`/`y`/`xend`/`yend` columns as lines beneath the node points — low `alpha` keeps a dense graph legible.

Well-known authors of each era — Shakespeare, Chaucer, Swift — sit near the center; lesser-known pages are pushed to the edges.

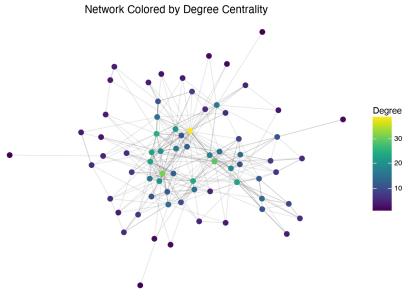


Undirected link network among 75 author pages.

Degree Centrality

The simplest centrality score: a node's *degree* is just its number of neighbors, counting every edge it touches (not just incoming links, as in the raw counts above).

Nodes with the highest degree cluster near the middle of the layout — the two ideas agree, but degree only sees a node's *direct* neighbors.



Network colored by degree centrality.

Eigenvalue Centrality

A more holistic score: a node's importance is proportional to the importance of *its* neighbors, not just their count.

- $s_j = \lambda \sum_{i \in \text{Neighbors}(j)} s_i$ — a node scores high if it connects to other high-scoring nodes, scaled so the largest score is 1
- Only defined within a connected component; disconnected networks get a separate solution per component
- Concentrates score more sharply on the most central cluster than degree does, on a linear color scale

Closeness Centrality

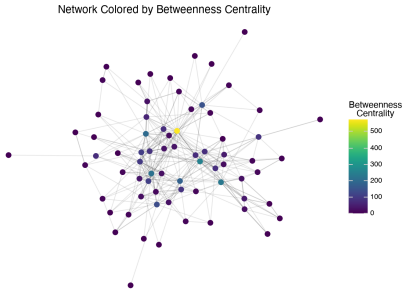
A score built from distance rather than direct connections: how many hops does it take, on average, to reach every other node?

- For each node, sum the reciprocals of its shortest-path lengths to every other node in its component
- High closeness means a node can reach the rest of the network quickly — useful for questions about spreading information efficiently
- Transitions more smoothly from central to peripheral nodes than eigenvalue centrality does

Betweenness Centrality

For every pair of nodes, count the shortest paths between them, then ask how many of those paths pass through a given node.

High betweenness marks *bridges* between otherwise separate parts of the network — a different notion of importance than "central within a cluster."



Network colored by betweenness centrality.

Clusters

Splitting nodes into groups so that most edges fall *within* a group rather than across groups — computed automatically by `DSNetwork.process` into a `cluster` column.

- Coloring the layout by cluster shows detected communities directly, but large networks quickly run out of plotting space for labels
- A table alternative scales better: `group_by(cluster).agg(members = id.str.concat("; "))` lists who belongs to each community
- Our 75-author network mostly forms one large cluster around a few well-known figures — larger, sparser networks show richer structure

Co-citation Networks

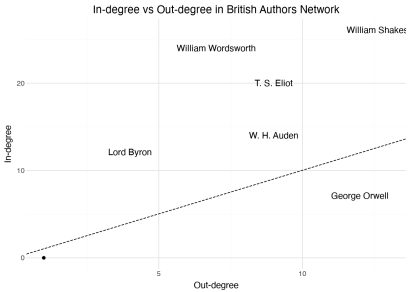
Direct citation links are sensitive to document length and publication order. A *co-citation* instead links two pages whenever a *third* page cites both — evidence they are related even with no direct link.

- Built the same way as any other network: filter an edge list (here, pairs cited together at least 6 times), then call `DSNetwork.process`
- Co-citation is undirected by definition — there's no notion of "which page cited first"
- Reshuffles the centrality rankings: John Milton tops the co-citation list despite not topping the raw citation count, reflecting shared thematic company rather than direct links

Directed Networks

Setting `directed=True` keeps the direction of each edge and splits degree into `degree_out` and `degree_in` (closeness centrality is no longer defined).

Most authors track the diagonal, but Orwell has far more outgoing links than incoming — his page cites influences that predate him and don't cite back.



In-degree vs. out-degree, with several authors highlighted.

Distance and Nearest-Neighbor Networks

Not every network comes from explicit citations. The angle distances from the previous chapter can define edges too: link two documents whenever their distance falls below a threshold.

- A *distance network* reveals implicit similarity in content, distinct from explicit reference structure
- A *symmetric nearest-neighbor network* links two documents only when each is among the other's k nearest neighbors — degree is capped at k, so no node can dominate
- Avoids piling every connection onto a few popular nodes; often produces more even, interpretable clusters than citation-based networks

From Text to Networks, and Beyond

- An edge list plus `DSNetwork.process` gives layout coordinates and four complementary centrality scores: degree, eigenvalue, closeness, betweenness
- Citation, co-citation, directed, distance, and nearest-neighbor networks all reuse the same tools — what changes is how the edge list is built
- Networks pair naturally with the vector distances from the previous chapter: geometry defines edges, and network tools describe the resulting structure

Outline

Image Data

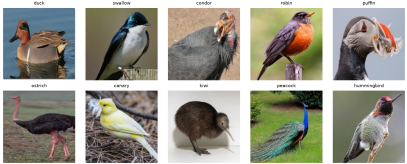
A digital image is an array of numbers: height $\times$ width $\times$ color channels, each entry an intensity from 0 to 255. The challenge is bridging raw pixel values and semantic meaning — getting from a grid of numbers to "this is a robin."

- OpenCV's `cv2.imread` loads an image as a NumPy array, in BGR (not RGB) channel order by convention
- Hand-computed pixel statistics (brightness, color) are fast and interpretable but describe the whole frame, not just the subject
- Pre-trained neural networks solve the semantic gap the same way TF-IDF vectors did for text: by turning each image into a point in space

The Birds Dataset

Ten species, preprocessed to a standard size — a table of `filepath` and `label` columns pointing to files stored separately from the metadata, a common pattern for image data.

Some species (canary, peacock) have distinctive, near-unmistakable coloration; others (duck, swallow, robin) are easy to confuse from color alone.



One example image per species in the birds dataset.

Brightness: A First Pixel Statistic

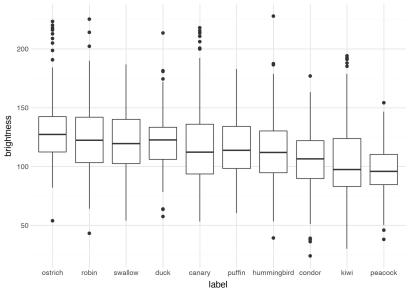
The simplest possible image summary: the mean of every pixel value across the array.

- Higher mean ⇒ a brighter image overall; lower mean ⇒ a darker one — computed once per image in a simple loop over the dataset
- The darkest images are dominated by black backgrounds, not dark birds; the brightest have light backgrounds, not light birds
- A pixel statistic describes the whole photograph — subject, background, and lighting all mixed together

Brightness Overlaps Across Species

Ostriches (bright savanna shots) sit at the top; peacocks (dramatic dark backgrounds) sit at the bottom — but every species' box overlaps most of the others.

Brightness reflects photographic context as much as the bird itself, so it is a weak signal for telling species apart.



Brightness distribution by species, heavily overlapping.

Hue, Saturation, and Value

RGB is not how humans perceive color. HSV separates a pixel into three more intuitive components, computed by OpenCV in one call (`cv2.cvtColor(img, cv2.COLOR_BGR2HSV)`).

- *Hue*: the pure color as an angle on a color wheel (red 0°, green 120°, blue 240°)
- *Saturation*: how vivid vs. washed-out/gray the color is; *Value*: how light vs. dark it is
- `DSImage.compute_colors` bins hue (plus saturation/value for black, white, gray) into named color categories per pixel, giving proportions like “18% yellow”

Color Proportions Across Species

Per-image color proportions, computed the same way as brightness, reveal cleaner patterns than brightness alone.

- Canaries lead in yellow content, as expected, but so do some hummingbirds photographed against yellow-green foliage
- Peacocks lead in cyan — their iridescent blue-green plumage — with swallows close behind, mostly from sky in the background rather than the bird
- Still a scene-level statistic: useful for spotting outliers and broad patterns, not for telling a duck from a robin

Transfer Learning

No small set of hand-written rules captures how pixels are *arranged* into an eye, a wing, a beak. Deep neural networks trained on millions of labeled images learn that arrangement directly.

- Early layers detect edges and textures; later layers combine them into larger structures; the final layer predicts a category
- Just before that last step, the network has reduced the image to a fixed-length vector — an *embedding*, in exactly the sense of the previous chapter's document vectors
- *Transfer learning*: reuse a model trained on someone else's categories, keeping only the embedding — generic enough to describe any image

Embedding Images with ViT

`ViTEmbedder` wraps Google's Vision Transformer, trained on 14 million images across 21,843 classes — it splits an image into patches and treats each one the way a language model treats a word.

Every image becomes a dense 768-number vector; unlike sparse TF-IDF, nearly every entry is non-zero and no single dimension has a name. Only the geometry matters.

```
vit = ViTEmbedder()
birds.select(c.filepath, c.label, c.vit)
```

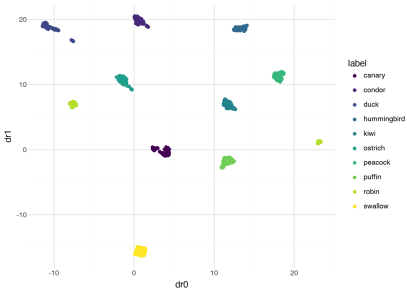
```
shape: (1555, 3)
filepath                    label     vit
"media/birds10/00000.png"   "canary"  [0.0179, -0.0305, 0.0577, ...]
"media/birds10/00001.png"   "canary"  [0.0248, -0.0453, 0.0796, ...]
"media/birds10/00002.png"   "canary"  [0.0506, -0.0245, 0.0628, ...]
...                         ...       ...

# each vit entry is a length-768 list
```

UMAP of ViT Embeddings

Projecting the 768-dimensional ViT embeddings to two dimensions with UMAP separates all ten species almost perfectly.

Nothing in the code told the model about our species — it was trained on entirely different images and labels. Compare this to the heavily overlapping brightness boxplot: the embedding encodes far more of what the image actually shows.



UMAP projection of ViT embeddings, colored by species.

Zero-Shot Classification with SigLIP

SigLIP embeds images *and* text into the same shared space — an image and its caption land close together, with no task-specific training required.

- Similarity is measured by the dot product; since SigLIP embeddings are unit length, this is exactly the cosine similarity used for document vectors
- Embed the phrase “a photo of a canary” and every image’s similarity to it can be computed with a single `dot_product` column
- Classify by comparing each image to a *text* embedding for every species label, and picking the closest match — no labeled training examples at all

Zero-Shot Results

- Cross-joining every image against every species' text embedding and keeping the highest similarity per image reproduces the true label almost every time
- Achieved with zero labeled training examples — valuable when labels are expensive, categories are numerous, or a classifier needs to be prototyped quickly
- The few errors involve birds in unusual poses or cropped at the frame's edge, cutting off the features that distinguish one species from another

From Pixels to Vectors

- Pixel statistics (brightness, color proportions) are fast, interpretable, and describe the whole frame — background and lighting included
- Pre-trained embeddings close the semantic gap: ViT for general visual similarity, SigLIP for connecting images directly to plain-language descriptions
- The same theme runs through the last three chapters: once documents, networks, or images become vectors or graphs, one small geometric toolkit — distance, similarity, projection — does most of the analytical work